# Understanding Influence Functions: From Edge Edits in Graphs to Group Influence in LLMs

Dongwoo Kim

Visiting Fellow, School of Computing, Australian National University
Associate Professor, POSTECH

April 2026

**“What would have happened without this data point ?”**

# Motivation: The "Leave-One-Out" Problem

**Core Question:** How does a single training data point affect our model's predictions?

# Motivation: The "Leave-One-Out" Problem

**Core Question:** How does a single training data point affect our model's predictions?

Imagine a scenario where we want to understand a model's behavior, find mislabeled data, or defend against data poisoning attacks.

# Motivation: The "Leave-One-Out" Problem

**Core Question:** How does a single training data point affect our model's predictions?

Imagine a scenario where we want to understand a model's behavior, find mislabeled data, or defend against data poisoning attacks.

**The Naive Approach (Leave-One-Out (LOO) Retraining):**

- Remove the target data point $z$ from the training set.
- Retrain the model entirely from scratch.
- Compare the new parameters or predictions with the original ones.

# Motivation: The "Leave-One-Out" Problem

**Core Question:** How does a single training data point affect our model's predictions?

Imagine a scenario where we want to understand a model's behavior, find mislabeled data, or defend against data poisoning attacks.

**The Naive Approach (Leave-One-Out (LOO) Retraining):**

- Remove the target data point $z$ from the training set.
- Retrain the model entirely from scratch.
- Compare the new parameters or predictions with the original ones.

**The Problem:** Retraining a modern neural network for every single training point is computationally impossible. We need an efficient approximation.

# Mathematical Setup

Let $z = (x, y)$ be a training point and let $L(z, \theta)$ be the loss function parameterized by $\theta$.

The standard Empirical Risk Minimization (ERM) finds the optimal parameters $\hat{\theta}$ by minimizing the average loss over all $n$ training points:

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} L(z_i, \theta) \tag{1}$$

If we upweight a specific training point $z$ by an infinitesimally small amount $\epsilon$, the new objective becomes:

$$\hat{\theta}_{\epsilon,z} = \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} L(z_i, \theta) + \epsilon L(z, \theta) \tag{2}$$

# Deriving the Influence Function

Instead of retraining to find $\hat{\theta}_{\epsilon,z}$, we can approximate it using a first order Taylor expansion around $\hat{\theta}$.

Assuming the empirical risk is strictly convex and twice differentiable, we rely on the Hessian matrix $H_{\hat{\theta}}$:

$$H_{\hat{\theta}} = \frac{1}{n} \sum_{i=1}^{n} \nabla_{\theta}^{2} L(z_i, \hat{\theta}) \tag{3}$$

The **Influence Function** for the parameters, which measures the rate of change in $\hat{\theta}$ as we upweight $z$, is given by:

$$\mathcal{I}_{\text{up, params}}(z) = \left. \frac{d\hat{\theta}_{\epsilon,z}}{d\epsilon} \right|_{\epsilon=0} = -H_{\hat{\theta}}^{-1} \nabla_{\theta} L(z, \hat{\theta}) \tag{4}$$

# Influence on Test Predictions

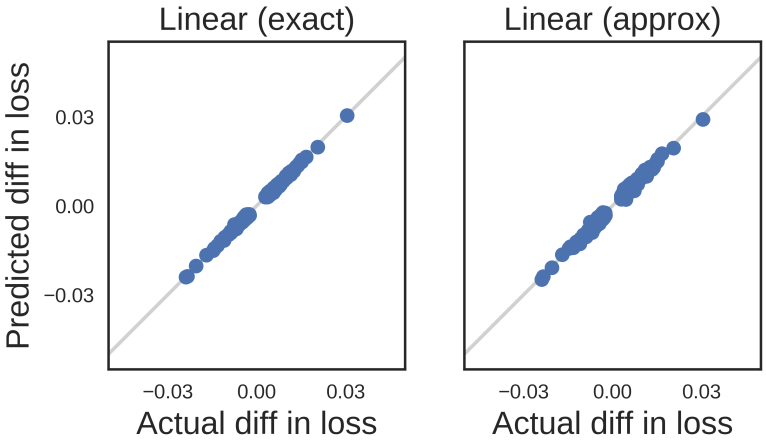
Often, we care more about how removing a training point $z$ affects the loss on a specific test point $z_{\text{test}}$.

Using the chain rule, we can compute the influence of upweighting $z$ on the test loss $L(z_{\text{test}}, \hat{\theta})$[cite: 89, 90]:

$$\begin{aligned}
\mathcal{I}_{\text{up, loss}}(z, z_{\text{test}}) &= \frac{d}{d\epsilon} L(z_{\text{test}}, \hat{\theta}_{\epsilon,z}) \Big|_{\epsilon=0} \\
&= \nabla_\theta L(z_{\text{test}}, \hat{\theta})^\top \frac{d\hat{\theta}_{\epsilon,z}}{d\epsilon} \Big|_{\epsilon=0} \\
&= -\nabla_\theta L(z_{\text{test}}, \hat{\theta})^\top H_{\hat{\theta}}^{-1} \nabla_\theta L(z, \hat{\theta}) \qquad (5)
\end{aligned}$$

Removing $z$ from the training set is approximated by setting $\epsilon = -1/n$[cite: 82, 92].

# Influence vs. Leave-One-Out Retraining




Correlation between influence function estimates and actual LOO retraining effects.

# Key Applications of Influence Functions

With this closed form approximation, we unlock several powerful capabilities in machine learning[cite: 861]:

1. **Model Interpretability:** We can explain a model's prediction on $z_{\text{test}}$ by tracing it back to the most influential training samples.
2. **Identifying Mislabeled Data:** Training points with highly negative influence on the validation set are often mislabeled or harmful to generalization.
3. **Adversarial Robustness:** Influence functions can be used to craft stronger data poisoning attacks or, conversely, to detect and sanitize poisoned data.

# Challenges in Modern Architectures

While elegant, applying traditional influence functions to deep neural networks presents significant hurdles:

- **Computational Cost:** Storing and inverting the exact Hessian matrix requires $O(d^3)$ operations, which is infeasible for millions of parameters[cite: 94, 938].
- **Non Convexity:** Neural networks have non convex loss landscapes[cite: 13, 27]. The Hessian is often singular or contains negative eigenvalues, violating the strict convexity assumption[cite: 93, 942].
- **Structural Complexity:** Standard influence functions assume independent, identically distributed data points.

# Influence Functions in Deep Learning

When we apply classical influence functions to deep neural networks, the theoretical guarantees often break down. Why?

- **Non-Convexity:** The loss landscape is highly non-convex. The Hessian $H_{\hat{\theta}}$ often has negative eigenvalues, meaning it is not Positive Definite and might not be invertible.
- **Non-Convergence:** We rarely train neural networks to a true local minimum (e.g., we use early stopping). Therefore, the assumption that the gradient $\nabla_\theta L(\hat{\theta}) = 0$ is violated.

**The Result:** In deep learning, standard influence function estimates often fail to accurately predict the true parameter changes of Leave-One-Out (LOO) retraining.

# Influence Functions in Practice

When applying classical influence functions to deep neural networks, the foundational assumptions break down:

- **Non Convergence:** We use early stopping or stochastic gradient descent (SGD), meaning our parameters $\tilde{\theta}$ are not the true global minimum ($\tilde{\theta} \neq \hat{\theta}$). The gradient is not zero.
- **Non Convexity:** Neural network loss landscapes are highly non convex. As a result, the Hessian evaluated at our current parameters, $H_{\tilde{\theta}}$, could have negative eigenvalues.

If the Hessian has negative eigenvalues, it is not positive definite, making the inverse $H_{\tilde{\theta}}^{-1}$ unstable or physically meaningless for optimization.

# Addressing Non Convexity and Non Convergence

To extract meaningful influence estimates despite these issues, Koh and Liang (2017) proposed forming a **convex quadratic approximation** of the loss around the current parameters $\tilde{\theta}$.

They approximate the loss $\tilde{L}(z, \theta)$ as:

$$\tilde{L}(z, \theta) = L(z, \tilde{\theta}) + \nabla L(z, \tilde{\theta})^\top (\theta - \tilde{\theta}) + \frac{1}{2}(\theta - \tilde{\theta})^\top (H_{\tilde{\theta}} + \lambda I)(\theta - \tilde{\theta}) \tag{6}$$

**The Damping Term ($\lambda I$):**

- If $H_{\tilde{\theta}}$ has negative eigenvalues, we add a scalar matrix $\lambda I$ where $\lambda$ is large enough to make $(H_{\tilde{\theta}} + \lambda I)$ strictly positive definite.
- This mathematically ensures the approximated loss surface is a convex bowl, allowing the influence function formulas to hold locally.

# From Quadratic Approximations to PBRF

While the damped quadratic approximation ($\lambda I$) makes the math solvable, it is still a blunt instrument for highly complex models.

Recent studies show that simply adding $\lambda I$ to non converged models means the influence function is actually approximating a different objective: the **Proximal Bregman Response Function (PBRF)**.

Instead of just predicting a theoretical "Leave One Out" retraining, the PBRF asks:

> *"What is the effect of removing $z$, if we heavily penalize the model for altering its current predictions on the rest of the dataset?"*

*This realization provides the perfect foundation for our work on Graph Neural Networks, where preserving the surrounding structural predictions is critical.*

# Addressing Non-Convexity: Damping and GNH

To make the math work in non-convex settings, we must force the Hessian to be invertible and positive definite.

1. **Damping:** We add a scalar matrix $\lambda I$ to the Hessian.

$$\tilde{H}_{\hat{\theta}} = H_{\hat{\theta}} + \lambda I \tag{7}$$

This is mathematically equivalent to adding an $L_2$ proximity penalty, forcing the new parameters to stay close to the old ones.

2. **Gauss-Newton Hessian (GNH):** We approximate the parameter Hessian using the Jacobian of the network outputs $J_{y\theta}$ and the loss-space Hessian $H_y$.

$$G = J_{y\theta}^{\top} H_y J_{y\theta} \tag{8}$$

$G$ is guaranteed to be positive semi-definite.

# What are Influence Functions Actually Computing?

If influence functions in deep learning do not predict LOO retraining from scratch, what do they predict?

Recent research shows that they actually approximate the **Proximal Bregman Response Function (PBRF)**.

Instead of asking "What happens if we retrain without $z$?", the PBRF asks:

> *"What is the effect of removing $z$, provided we penalize the model for changing its current parameters and for altering its current predictions on the rest of the data?"*

Recognizing this allows us to adapt influence functions for highly complex, non-convex, and structured architectures.